

RoboPet — Phase 1: The Living Pet

Data Persistence & the Pure-Dart Decay Engine (Implementation Guide)

Locked architecture (v1): Flutter + Flame · Isar (offline-first) · Riverpod (UI phase) · earned-only premiumGems (no IAP) · async-ghost PvP only · 2D sprites · iOS 15+ / Android 8.0+. Complete copy-paste implementation for Phase 1: Isar schema, the pure-Dart decay engine, the repository that wires them together, and a full unit-test suite. No UI — data + math only.

0. Target directory layout

Everything below lives under `lib/` and `test/`. The domain layer is pure Dart (no Flutter, no Isar imports) so it is trivially unit-testable and later portable to a Cloud Function for anti-cheat re-simulation.

```
lib/
  domain/                                # PURE DART -- no Flutter, no Isar
    clock.dart
    decay/
      care_stat.dart
      decay_config.dart
      reconcile_result.dart
      decay_engine.dart
  data/                                  # Persistence layer (Isar)
    isar_service.dart
    robot_factory.dart
    robot_repository.dart
  models/
    robot_instance.dart
    robot_instance.g.dart               # generated
    player_profile.dart
    player_profile.g.dart               # generated
test/
  decay_engine_test.dart
```

Dependency rule: `data/` may import `domain/`, never the reverse. The decay engine knows nothing about Isar — the repository maps Isar embedded objects to/from the engine pure value objects.

1. Dependencies & codegen

pubspec.yaml (relevant entries)

```
environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
```

```

    sdk: flutter
    isar: ^3.1.0
    isar_flutter_libs: ^3.1.0    # bundled native binaries
    path_provider: ^2.1.0       # DB directory on device

dev_dependencies:
  flutter_test:
    sdk: flutter
  test: ^1.25.0
  build_runner: ^2.4.0
  isar_generator: ^3.1.0

```

Isar uses code generation. After creating the model files (Section 2), generate the *.g.dart files:

```

dart run build_runner build --delete-conflicting-outputs
# or, while iterating:
dart run build_runner watch --delete-conflicting-outputs

```

2. Isar schema (data models)

lib/data/models/robot_instance.dart

Isar cannot store arbitrary-keyed maps cleanly, so the conceptual careStats and equippedModules maps from the TDD are modeled as @embedded objects with fixed fields. Frequently-queried scalars (isActive, instanceId) are top-level and indexed.

```

import 'package:isar/isar.dart';

part 'robot_instance.g.dart';

@collection
class RobotInstance {
  Id id = Isar.autoIncrement; // Isar's internal PK (int)

  /// Stable, game-level identity. Distinct from Isar's [id].
  @Index(unique: true, replace: true)
  late String instanceId;

  late String speciesId; // FK -> RobotSpecies definition (static content)
  String nickname = '';

  @Enumerated(EnumType.name) // store as string -> safe to reorder enum
  late Rarity rarity;

  @Enumerated(EnumType.name)
  late RobotClass robotClass; // 'class' is a reserved word in Dart

  int level = 1;
  int xp = 0;
  int xpToNext = 100;

  // --- Care stats (these decay over time) ---
  CareStatData battery = CareStatData();
  CareStatData oil = CareStatData();
  CareStatData entertainment = CareStatData();

  // --- Base combat stats (pre-module). Used from Phase 3 onward. ---
  CombatStatsData combat = CombatStatsData();

  // --- Equipped modules (fixed slot set) ---
  Loadout loadout = Loadout();

```

```

List<String> abilities = [];

// --- Timestamps (ALWAYS stored/treated as UTC) ---
late DateTime createdAt;

/// Drives offline reconciliation. The single source of truth for "when did
/// we last compute this robot's state".
late DateTime lastTickAt;

DateTime? lastFedAt;

// --- Flags ---
@index()
bool isActive = false; // the currently focused pet
bool isFavorite = false;
int neglectStrikes = 0;

/// Idle currency generation rate (scrap/second). Later derived from
/// level/modules; a stored field for Phase 1.
double scrapPerSecond = 0.05;

int schemaVersion = 1; // bump + migrate on schema changes
}

@embedded
class CareStatData {
  double value = 0;
  double max = 100;
  double drainPerSecond = 0;
}

@embedded
class CombatStatsData {
  int hp = 1;
  int attack = 0;
  int defense = 0;
  int speed = 0;
  double critChance = 0;
}

@embedded
class Loadout {
  String? wheels;
  String? armor;
  String? sensor;
  String? core;
  String? utility;
}

enum Rarity { common, rare, epic, legendary }

enum RobotClass { scout, tank, brawler, support }

```

Isar + DateTime: Isar stores DateTime in UTC internally but returns it in the device local time on read. The engine defensively calls `.toUtc()` on every timestamp, so the math is correct regardless — but always write UTC (`DateTime.now().toUtc()`) to keep your data clean.

lib/data/models/player_profile.dart

```

import 'package:isar/isar.dart';

part 'player_profile.g.dart';

@collection

```

```

class PlayerProfile {
  Id id = Isar.autoIncrement;

  @Index(unique: true, replace: true)
  late String playerId; // Firebase UID (or a local uuid pre-auth)

  String displayName = '';

  Currencies currencies = Currencies();
  LootBoxes lootBoxes = LootBoxes();

  List<String> ownedRobotIds = [];
  List<String> ownedModuleIds = [];

  String? activeRobotId;

  /// When the next free loot box unlocks (UTC). Null = none pending.
  DateTime? nextFreeBoxAt;

  int schemaVersion = 1;
}

@embedded
class Currencies {
  int scrap = 0;

  /// v1: EARNED IN-GAME ONLY. No IAP wiring in Phase 1.
  int premiumGems = 0;

  int rareCogs = 0;
}

@embedded
class LootBoxes {
  int common = 0;
  int rare = 0;
  int epic = 0;
}

```

lib/data/isar_service.dart

```

import 'package:isar/isar.dart';
import 'package:path_provider/path_provider.dart';

import 'models/player_profile.dart';
import 'models/robot_instance.dart';

/// Opens (and caches) the single Isar instance for the app.
class IsarService {
  IsarService._();

  static Isar? _isar;

  static Future<Isar> instance() async {
    if (_isar != null && _isar!.isOpen) return _isar!;
    final dir = await getApplicationDocumentsDirectory();
    _isar = await Isar.open(
      [RobotInstanceSchema, PlayerProfileSchema],
      directory: dir.path,
      inspector: true, // enables the Isar Inspector in debug builds
    );
    return _isar!;
  }
}

```

3. The pure-Dart decay engine

This is the heart of Phase 1. It is pure (deterministic, no `DateTime.now()` inside, no I/O), so the same code runs on-resume (one big jump) and on each foreground frame (tiny jumps), and can be re-run server-side later.

`lib/domain/clock.dart`

```
/// A source of "now". Injected so time is controllable in tests.
abstract class Clock {
  DateTime now();
}

class SystemClock implements Clock {
  const SystemClock();
  @override
  DateTime now() => DateTime.now().toUtc();
}

/// Test double: a clock you can set and advance deterministically.
class FixedClock implements Clock {
  FixedClock(this._now);
  DateTime _now;

  void set(DateTime t) => _now = t.toUtc();
  void advance(Duration d) => _now = _now.add(d);

  @override
  DateTime now() => _now.toUtc();
}
```

`lib/domain/decay/care_stat.dart`

```
/// Immutable value object for a single decaying care stat.
class CareStat {
  final double value;
  final double max;
  final double drainPerSecond;

  const CareStat({
    required this.value,
    required this.max,
    required this.drainPerSecond,
  }) : assert(max > 0, 'max must be positive'),
       assert(drainPerSecond >= 0, 'drain must be non-negative');

  /// Value after [seconds] of decay, clamped to [0, max].
  double valueAfter(double seconds) =>
    (value - drainPerSecond * seconds).clamp(0.0, max).toDouble();

  /// Seconds until this stat reaches zero from its current value.
  /// Returns null if it never empties (no drain), 0 if already empty.
  double? secondsUntilEmpty() {
    if (drainPerSecond <= 0) return null;
    if (value <= 0) return 0;
    return value / drainPerSecond;
  }

  CareStat copyWith({double? value, double? max, double? drainPerSecond}) =>
    CareStat(
      value: value ?? this.value,
      max: max ?? this.max,
      drainPerSecond: drainPerSecond ?? this.drainPerSecond,
    );
}
```

lib/domain/decay/decay_config.dart

```
/// Tunable knobs for offline progression. In production these come from
/// Firebase Remote Config; here they have sensible defaults.
class DecayConfig {
  /// Decay stops accruing after this much offline time (grace period),
  /// so a player returning from vacation isn't punished with a dead robot.
  /// null = decay applies for the full elapsed duration.
  final Duration? decayCap;

  /// Idle currency only accrues up to this much offline time.
  /// null = currency accrues for the full elapsed duration.
  final Duration? idleEarnCap;

  const DecayConfig({
    this.decayCap = const Duration(hours: 12),
    this.idleEarnCap = const Duration(hours: 8),
  });

  /// No caps -- useful for tests and pure linear-decay reasoning.
  static const DecayConfig unlimited =
    DecayConfig(decayCap: null, idleEarnCap: null);
}
```

lib/domain/decay/reconcile_result.dart

```
import 'care_stat.dart';

/// The immutable output of a reconciliation pass. The repository applies
/// these values back onto the persisted model.
class ReconcileResult {
  final Map<String, CareStat> stats; // updated stat values
  final Duration elapsed; // raw wall-clock time since lastTickAt
  final Duration decayApplied; // elapsed, capped by decayCap
  final Duration earnWindow; // elapsed, capped by idleEarnCap
  final int scrapEarned; // whole units of scrap earned away
  final Set<String> depletedStats; // stats that hit zero during the gap
  final DateTime lastTickAt; // new lastTickAt to persist (UTC)

  const ReconcileResult({
    required this.stats,
    required this.elapsed,
    required this.decayApplied,
    required this.earnWindow,
    required this.scrapEarned,
    required this.depletedStats,
    required this.lastTickAt,
  });

  bool get neglected => depletedStats.isNotEmpty;
  bool get isNoOp => elapsed <= Duration.zero;
}
```

lib/domain/decay/decay_engine.dart

```
import 'care_stat.dart';
import 'decay_config.dart';
import 'reconcile_result.dart';

/// Pure, deterministic timestamp-delta simulation.
///
/// Given the previous tick time, the current time, and the stats, it returns
/// the new stat values + currency earned. Identical results whether called
/// once over a long gap (offline catch-up) or repeatedly over short gaps
/// (foreground smoothing).
class DecayEngine {
  final DecayConfig config;
}
```

```

const DecayEngine([this.config = const DecayConfig()]);

ReconcileResult reconcile({
  required DateTime lastTickAt,
  required DateTime now,
  required Map<String, CareStat> stats,
  double scrapPerSecond = 0.0,
}) {
  final lastUtc = lastTickAt.toUtc();
  final nowUtc = now.toUtc();
  final rawElapsed = nowUtc.difference(lastUtc);

  // Monotonic guard: never advance backwards (clock rollback / TZ change).
  if (rawElapsed <= Duration.zero) {
    return ReconcileResult(
      stats: Map<String, CareStat>.unmodifiable(stats),
      elapsed: Duration.zero,
      decayApplied: Duration.zero,
      earnWindow: Duration.zero,
      scrapEarned: 0,
      depletedStats: const <String>{},
      lastTickAt: lastUtc, // unchanged
    );
  }

  final decayWindow = _cap(rawElapsed, config.decayCap);
  final earnWindow = _cap(rawElapsed, config.idleEarnCap);
  final decaySeconds = _toSeconds(decayWindow);

  final newStats = <String, CareStat>{};
  final depleted = <String>{};
  stats.forEach((key, stat) {
    final emptyAt = stat.secondsUntilEmpty();
    if (emptyAt != null && emptyAt <= decaySeconds) depleted.add(key);
    newStats[key] = stat.copyWith(value: stat.valueAfter(decaySeconds));
  });

  final scrapEarned = (scrapPerSecond * _toSeconds(earnWindow)).floor();

  return ReconcileResult(
    stats: Map<String, CareStat>.unmodifiable(newStats),
    elapsed: rawElapsed,
    decayApplied: decayWindow,
    earnWindow: earnWindow,
    scrapEarned: scrapEarned,
    depletedStats: Set<String>.unmodifiable(depleted),
    lastTickAt: nowUtc,
  );
}

static Duration _cap(Duration value, Duration? cap) =>
  cap == null ? value : (value < cap ? value : cap);

static double _toSeconds(Duration d) =>
  d.inMicroseconds / Duration.microsecondsPerSecond;
}

```

4. Wiring it together (repository + factory)

`lib/data/robot_factory.dart`

```
import 'models/robot_instance.dart';
```

```

class RobotFactory {
  /// A fresh, fully-charged starter rover. [now] should be UTC.
  static RobotInstance starter({
    required String instanceId,
    required DateTime now,
    String speciesId = 'rover_scout',
    String nickname = 'Sparky',
    bool isActive = true,
  }) {
    final utc = now.toUtc();
    return RobotInstance()
      ..instanceId = instanceId
      ..speciesId = speciesId
      ..nickname = nickname
      ..rarity = Rarity.common
      ..robotClass = RobotClass.scout
      ..battery = (CareStatData()
        ..value = 100
        ..max = 100
        ..drainPerSecond = 0.0008) // ~ -2.9/hour
      ..oil = (CareStatData()
        ..value = 100
        ..max = 100
        ..drainPerSecond = 0.0005)
      ..entertainment = (CareStatData()
        ..value = 100
        ..max = 100
        ..drainPerSecond = 0.0011)
      ..combat = (CombatStatsData()
        ..hp = 100
        ..attack = 20
        ..defense = 15
        ..speed = 18)
      ..createdAt = utc
      ..lastTickAt = utc
      ..isActive = isActive
      ..scrapPerSecond = 0.05
      ..schemaVersion = 1;
  }
}

```

lib/data/robot_repository.dart

The bridge: load -> reconcile via the pure engine -> persist. Both loadActiveAndReconcile() and feed() funnel through one private _reconcileInPlace, so there is exactly ONE path that advances time.

```

import 'package:isar/isar.dart';

import '../domain/clock.dart';
import '../domain/decay/care_stat.dart';
import '../domain/decay/decay_engine.dart';
import '../domain/decay/reconcile_result.dart';
import 'models/player_profile.dart';
import 'models/robot_instance.dart';

class RobotRepository {
  final Isar _isar;
  final DecayEngine _engine;
  final Clock _clock;

  RobotRepository(
    this._isar, {
    DecayEngine engine = const DecayEngine(),
    Clock clock = const SystemClock(),
  }) : _engine = engine,
      _clock = clock;
}

```



```

    /// Loads the active robot, applies offline catch-up, persists, returns it.
    Future<RobotInstance?> loadActiveAndReconcile() async {
        final robot =
            await _isar.robotInstances.filter().isActiveEqualTo(true).findFirst();
        if (robot == null) return null;
        final result = _reconcileInPlace(robot);
        await _persist(robot, result);
        return robot;
    }

    /// In-app care action: settle elapsed decay first, THEN add charge, so the
    /// boost stacks on the correct (decayed) base value.
    Future<RobotInstance?> feed(String instanceId, {double amount = 60}) async {
        final robot = await _isar.robotInstances
            .filter()
            .instanceIdEqualTo(instanceId)
            .findFirst();
        if (robot == null) return null;

        final result = _reconcileInPlace(robot);
        robot.battery.value =
            (robot.battery.value + amount).clamp(0.0, robot.battery.max).toDouble();
        robot.lastFedAt = _clock.now().toUtc();
        await _persist(robot, result);
        return robot;
    }

    // --- internals ---

    ReconcileResult _reconcileInPlace(RobotInstance r) {
        final result = _engine.reconcile(
            lastTickAt: r.lastTickAt,
            now: _clock.now(),
            stats: {
                'battery': _toDomain(r.battery),
                'oil': _toDomain(r.oil),
                'entertainment': _toDomain(r.entertainment),
            },
            scrapPerSecond: r.scrapPerSecond,
        );

        if (!result.isNoOp) {
            r.battery.value = result.stats['battery']!.value;
            r.oil.value = result.stats['oil']!.value;
            r.entertainment.value = result.stats['entertainment']!.value;
            r.lastTickAt = result.lastTickAt;
            if (result.neglected) r.neglectStrikes += 1;
        }
        return result;
    }

    Future<void> _persist(RobotInstance robot, ReconcileResult result) async {
        await _isar.writeTxn(() async {
            await _isar.robotInstances.put(robot);
            if (result.scrapEarned > 0) {
                final profile = await _isar.playerProfiles.where().findFirst();
                if (profile != null) {
                    profile.currencies.scrap += result.scrapEarned;
                    await _isar.playerProfiles.put(profile);
                }
            }
        });
    }

    CareStat _toDomain(CareStatData d) => CareStat(
        value: d.value,
        max: d.max,
        drainPerSecond: d.drainPerSecond,
    );

```

```
}
```

5. Unit-testable approach

Because the domain layer has zero Flutter/Isar imports, the test suite runs with plain dart test (no emulator, no mocking). You feed the engine explicit timestamps; for the repository layer, inject a FixedClock to control "now".

test/decay_engine_test.dart

```
import 'package:test/test.dart';

import 'package:robopet/domain/decay/care_stat.dart';
import 'package:robopet/domain/decay/decay_config.dart';
import 'package:robopet/domain/decay/decay_engine.dart';

void main() {
  final base = DateTime.utc(2026, 1, 1, 12, 0, 0);

  // Round numbers for easy mental math: battery drains 1/s, oil 0.5/s.
  Map<String, CareStat> freshStats() => {
    'battery': const CareStat(value: 100, max: 100, drainPerSecond: 1.0),
    'oil': const CareStat(value: 50, max: 100, drainPerSecond: 0.5),
  };

  group('DecayEngine.reconcile', () {
    test('applies linear decay over elapsed time', () {
      const engine = DecayEngine(DecayConfig.unlimited);
      final r = engine.reconcile(
        lastTickAt: base,
        now: base.add(const Duration(seconds: 10)),
        stats: freshStats(),
      );
      expect(r.stats['battery']!.value, closeTo(90, 1e-9));
      expect(r.stats['oil']!.value, closeTo(45, 1e-9));
      expect(r.elapsed, const Duration(seconds: 10));
      expect(r.neglected, isFalse);
    });

    test('clamps at zero -- never goes negative', () {
      const engine = DecayEngine(DecayConfig.unlimited);
      final r = engine.reconcile(
        lastTickAt: base,
        now: base.add(const Duration(seconds: 1000)),
        stats: freshStats(),
      );
      expect(r.stats['battery']!.value, 0);
      expect(r.stats['oil']!.value, 0);
    });

    test('monotonic guard: a backwards clock is a no-op', () {
      const engine = DecayEngine(DecayConfig.unlimited);
      final r = engine.reconcile(
        lastTickAt: base,
        now: base.subtract(const Duration(hours: 5)),
        stats: freshStats(),
      );
      expect(r.isNoOp, isTrue);
      expect(r.stats['battery']!.value, 100);
      expect(r.lastTickAt, base); // did NOT move backwards
    });
  });
}
```

```

test('decay cap (grace period) freezes decay after the cap', () {
  const engine = DecayEngine(
    DecayConfig(decayCap: Duration(seconds: 30), idleEarnCap: null));
  final r = engine.reconcile(
    lastTickAt: base,
    now: base.add(const Duration(hours: 10)),
    stats: freshStats(),
  );
  // Only 30s of decay applied: 100 - 1*30 = 70.
  expect(r.stats['battery']!.value, closeTo(70, 1e-9));
  expect(r.decayApplied, const Duration(seconds: 30));
});

test('idle earnings are capped by idleEarnCap', () {
  const engine = DecayEngine(
    DecayConfig(decayCap: null, idleEarnCap: Duration(hours: 8)));
  final r = engine.reconcile(
    lastTickAt: base,
    now: base.add(const Duration(hours: 24)),
    stats: freshStats(),
    scrapPerSecond: 1.0, // 1 scrap/second
  );
  expect(r.scrapEarned, 8 * 3600); // capped to 8 hours
  expect(r.earnWindow, const Duration(hours: 8));
});

test('flags neglect when a stat depletes during the gap', () {
  const engine = DecayEngine(DecayConfig.unlimited);
  final r = engine.reconcile(
    lastTickAt: base,
    now: base.add(const Duration(seconds: 200)),
    stats: freshStats(), // oil empties @100s, battery @100s
  );
  expect(r.neglected, isTrue);
  expect(r.depletedStats, containsAll(<String>['battery', 'oil']));
});

test('foreground ticks == one offline catch-up (determinism)', () {
  const engine = DecayEngine(DecayConfig.unlimited);

  final once = engine.reconcile(
    lastTickAt: base,
    now: base.add(const Duration(seconds: 10)),
    stats: freshStats(),
  );

  var current = freshStats();
  var t = base;
  for (var i = 0; i < 10; i++) {
    final next = t.add(const Duration(seconds: 1));
    final r = engine.reconcile(lastTickAt: t, now: next, stats: current);
    current = Map<String, CareStat>.of(r.stats);
    t = next;
  }

  expect(current['battery']!.value,
    closeTo(once.stats['battery']!.value, 1e-9));
  expect(current['oil']!.value, closeTo(once.stats['oil']!.value, 1e-9));
});
}

```

Repository tests (Isar) use `Isar.open(..., directory: tempDir)` plus an injected `FixedClock`: `final repo = RobotRepository(isar, clock: FixedClock(base));` then `clock.advance(Duration(hours: 3));` and assert the persisted robot stats. No real waiting, no flaky time-based tests.

Run the pure tests (fast, no device):

```
dart test test/decay_engine_test.dart
```

6. Phase 1 "done" checklist & next step

- Models compile and build_runner generates *.g.dart.
- IsarService.instance() opens the DB on device.
- Seed one robot via RobotFactory.starter(...) + a PlayerProfile.
- All 7 engine tests pass with dart test.
- Manual check: seed -> set lastTickAt an hour in the past -> loadActiveAndReconcile() -> stats reflect ~1h of decay and scrap accrued.

Next (still Phase 1): wire a Riverpod StreamProvider on top of isar.robotInstances.watchObject(id) plus an app-lifecycle observer that calls loadActiveAndReconcile() on resume — then the minimal UI (stat bars + feed/clean/play buttons).